

IoT Assignment 02-07-2026

Mishthi Chopra - IoT025

Q1) You are building a smart irrigation system that must monitor soil moisture, control a water pump, and send data to a mobile application every 5 seconds. You have both an Arduino UNO and an ESP32. Which controller would you choose and why? Would your answer change if internet connectivity was not required?

For this scenario, I would choose the ESP32 because it has built-in Wi-Fi and Bluetooth, making it suitable for IoT applications where data must be sent to a mobile application regularly.

Reasons:

- Built-in Wi-Fi and Bluetooth
- Higher processing power
- Multiple GPIO pins and peripherals
- Low power consumption
- Scalability

If internet connectivity is not required, then the Arduino Uno would be sufficient for this task.

Reasons:

- Simple to program and debug.
- Adequate processing capability for sensor monitoring and pump control.
- Lower cost for basic automation projects.
- Large community support and extensive libraries.

Q2) A smart factory contains 500 sensors distributed across a 2 km area. Compare Wi-Fi, Bluetooth, LoRa, and MQTT-based communication. Which protocol or combination of protocols would you choose and why?

Protocol	Advantages	Limitations
Wi-Fi	High data rate, internet connectivity	Short range, high power consumption
Bluetooth	Low power, low cost	Very short range, limited devices
LoRa	Long range (up to several km), very low power	Low data rate
MQTT	Lightweight messaging protocol for IoT, reliable data transfer	Requires an underlying network (e.g., Wi-Fi or LoRa)

For a smart factory with 500 sensors spread over 2 km, I would use LoRa + MQTT. LoRa provides long-range, low-power communication, making it ideal for covering the entire factory with minimal infrastructure. MQTT can then be used to send the collected sensor data reliably from a LoRa gateway to the cloud or monitoring application. This combination is scalable, energy-efficient, and well-suited for industrial IoT systems.

Q3) A temperature sensor suddenly starts showing impossible values (e.g., -120°C or 300°C). How you would determine whether the problem lies in the sensor, wiring, power supply, ADC, or software.

To identify the cause of impossible temperature readings, I would troubleshoot each component step by step:

- **Sensor:** Test with another temperature sensor or compare readings with a known thermometer.
- **Wiring:** Check for loose connections, damaged wires, or incorrect pin connections.
- **Power Supply:** Measure the supply voltage to ensure the sensor is receiving the correct power.
- **ADC:** Verify the ADC readings using a known input voltage to check if it is functioning properly.
- **Software:** Review the code for errors in sensor calibration, ADC conversion, or data processing.

By testing each component individually, the faulty part can be identified and replaced or corrected, ensuring accurate temperature measurements.

Q4) An ESP32-based weather station must operate on battery power for six months without maintenance. What hardware and software modifications would you make to maximize battery life?

To maximize battery life for six months, I would make the following changes:

- Use deep sleep mode so the ESP32 wakes only to read sensors and transmit data.
- Increase the interval between measurements and data transmission if possible.
- Use low-power sensors and efficient voltage regulators.
- Turn off unused peripherals such as Wi-Fi, Bluetooth, and LEDs when not needed.
- Optimize the code to reduce processing time and power consumption.
- Use a high-capacity battery or add a small solar panel for recharging.

Q5) An automatic braking system in a vehicle requires communication with almost zero delay. Which communication method would you choose and why?

For an automatic braking system, I would choose Controller Area Network (CAN) (CAN Bus). This is because it:

- Provides very low latency and real-time communication.
- Highly reliable, even in noisy automotive environments.
- Supports fast communication between sensors, controllers, and actuators.
- Includes error detection and fault handling for improved safety.

Q6) You need to measure the average temperature of a classroom using only one sensor. Where would you place the sensor and why? Explain how poor placement could affect the accuracy of your measurements. Assume the classroom to be a square-base cuboid of side length “a” and height “b”.

The sensor should be placed near the center of the classroom, at approximately half the room height ($b/2$), away from windows, doors, fans, air conditioners, heaters, and direct sunlight.

Reasons:

- The center represents the average air temperature of the room.
- Mid-height avoids hotter air near the ceiling and cooler air near the floor.
- Keeping it away from heat or airflow sources prevents biased readings.

Effect of poor placement:

- Near a window: Readings may be affected by sunlight or outside temperature.
- Near an AC or fan: Temperature may appear lower than the actual room average.
- Near the ceiling or floor: Readings may not represent the average due to temperature stratification.

Q7) Under what situations could an analog sensor perform better than a digital one?

An analog sensor can perform better than a digital sensor in the following situations:

- When continuous measurements are required without discrete steps.
- When fast response is needed, as analog sensors have minimal processing delay.
- In low-cost and simple circuits, where digital conversion is unnecessary.
- When the controller has a high-resolution ADC, allowing precise measurements.

Analog sensors are preferred for applications requiring continuous, real-time measurements with simple and cost-effective hardware, while digital sensors are better for noise immunity and direct digital communication.

Q8) Draw the complete architecture/block diagram of your IoT project, beginning from the sensor and ending at the user interface. Identify every possible point where a failure could occur and suggest one solution for each.

Architecture / Block Diagram

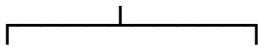
DHT22 + LDR



ESP32



Decision Logic



LEDs/Fan



Wi-Fi



Google Apps Script



Google Sheets



Dashboard / User

Possible Failures & Solutions

Failure Point	Solution
Sensor failure	Check wiring or replace sensor
ESP32 failure	Reset or reprogram ESP32
Wi-Fi disconnected	Auto reconnect to Wi-Fi
Google Apps Script error	Validate URL and redeploy script
Google Sheets not updating	Check internet and script permissions
LED/Fan not working	Check power supply and connections

Q9) If an attacker gains access to your IoT device through the Wi-Fi network, what kinds of attacks could they perform? Suggest at least five methods to improve the security of the system (with applicable solutions, do not cite far-fetched ideas).

If an attacker gains access to the IoT device through the Wi-Fi network, they could:

- Access or steal sensor data.
- Send fake commands to control the device.
- Modify or delete stored data.
- Install malicious software (malware).
- Make the device unavailable through a Denial-of-Service (DoS) attack.

Methods to improve security:

- Use strong Wi-Fi passwords and WPA2/WPA3 encryption.
- Enable user authentication with strong passwords or two-factor authentication.
- Encrypt data using HTTPS/TLS during communication.
- Keep the device firmware and software updated to fix security vulnerabilities.
- Restrict access using a firewall and secure network settings.

Q10) Your ESP32 must simultaneously interface with a DHT11 sensor, an ultrasonic sensor, an OLED display, a relay module, and a soil moisture sensor. What hardware or software conflicts could occur, and how would you solve them?

Possible conflicts:

- GPIO pin conflicts if multiple devices use the same pins.
- Timing conflicts between the DHT11 and ultrasonic sensor.
- Power issues if the relay draws high current.
- Communication conflicts if multiple devices share the same bus.

Solutions:

- Assign separate GPIO pins to each device.
- Use non-blocking code or a task scheduler.
- Power the relay using a separate supply if needed.
- Use I²C for the OLED and manage communication properly.

Q11) An ESP32 uploads sensor data to the cloud every minute. The internet connection fails for 12 hours. Design a strategy that ensures no important data is permanently lost.

If the internet fails for 12 hours:

- Store sensor readings in the ESP32's flash memory or an SD card.
- Add a timestamp to each reading.
- Retry connecting to the internet periodically.
- Once the connection is restored, upload all stored data in chronological order.
- Delete the local copy only after successful upload.

Q12) Your IoT device must operate continuously for five years without human intervention. What changes would you make to both the hardware and software to improve long-term reliability?

Hardware improvements:

- Use high-quality, industrial-grade components.
- Provide a stable regulated power supply with surge protection.
- Protect the device from dust, moisture, and heat using a proper enclosure.
- Use watchdog circuitry to recover from hardware failures.

Software improvements:

- Enable the watchdog timer to restart the system if it hangs.
- Handle errors and reconnect automatically after failures.
- Optimize memory usage to prevent memory leaks.
- Support secure firmware updates for maintenance.

Q13) Can an IoT system be considered "smart" if it does not use Artificial Intelligence? Justify your answer with suitable examples. What differentiates "smart" from "intelligent"?

Yes, an IoT system can be considered smart even without Artificial Intelligence.

A smart system can sense, process, and respond automatically based on predefined rules, while an intelligent system uses AI or machine learning to learn, adapt, and make decisions.

Example:

- A smart irrigation system that turns on a pump when soil moisture is low is smart.
- A system that learns weather patterns and predicts the best watering schedule is intelligent.

We can say that, all intelligent systems are smart, but not all smart systems require AI.

Q14) Compare polling and interrupt-based programming in embedded systems. In which situations would interrupts significantly improve the performance of an IoT device?

Polling

CPU continuously checks devices

Higher power consumption

Slower response to urgent events

Interrupts

CPU responds only when an event occurs

Lower power consumption

Fast response to critical events

Interrupts are useful when:

- Emergency events require immediate action.
- Sensors generate data at irregular intervals.
- Low power consumption is important.

Interrupt-based programming improves efficiency and responsiveness in most IoT applications.

Q15) Every engineering design involves trade-offs. Looking at your own IoT project, identify three design decisions where you sacrificed one aspect (such as cost, speed, power, accuracy, or complexity) to improve another. Explain whether you believe those trade-offs were justified.

Every engineering design involves balancing different factors. In our IoT project, we made the following trade-offs:

1. LEDs instead of Real Appliances

- Sacrificed: Real-world functionality
- Improved: Lower cost and safer testing
- Justification: LEDs allowed us to verify the automation logic without using high-voltage devices. Real appliances will be integrated in the future using relay modules.

2. Fixed Threshold Control instead of Variable Intensity Control

- Sacrificed: Precision and energy efficiency
- Improved: Simpler programming and faster implementation
- Justification: Fixed thresholds were sufficient for developing the basic automation system. Variable intensity control will be added in future versions.

3. DHT22 and LDR Sensors instead of More Advanced Sensors

- Sacrificed: Higher accuracy and additional environmental data
- Improved: Lower cost and easier implementation
- Justification: These sensors provide adequate readings for demonstrating the project's concept while keeping the prototype affordable.

Q16) Create a flowchart for an IoT system that stores sensor readings locally whenever Wi-Fi is unavailable and automatically uploads the stored data once the connection is restored. Mention all the programs/protocols/I-O used in the system.

Flowchart

Start

|

▼

Initialize ESP32, Sensors, Wi-Fi

|

▼

Read Sensor Data

|

▼

Is Wi-Fi Connected?

┌──────────────────┐

| Yes

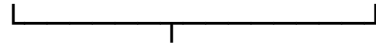
| No



Upload Data
to Cloud



Wait 1 Minute



Wi-Fi Restored?



Yes



Upload Stored Data



Delete Local Copy



Repeat



Store Data
in Flash/SD Card



Retry Wi-Fi



Programs/Protocols/I-O Used

- Controller: ESP32
- Input: Sensor (e.g., DHT11/Soil Moisture)
- Output: Cloud dashboard/mobile app
- Storage: ESP32 Flash Memory or SD Card
- Communication: Wi-Fi
- Protocol: MQTT or HTTP

Q17) Draw a flowchart showing how an ESP32 performs an OTA (Over-the-Air) firmware update, including error handling if the update fails.

Start



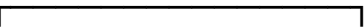
Connect to Wi-Fi



Check for New Firmware



Update Available?



No

Yes



Run Current

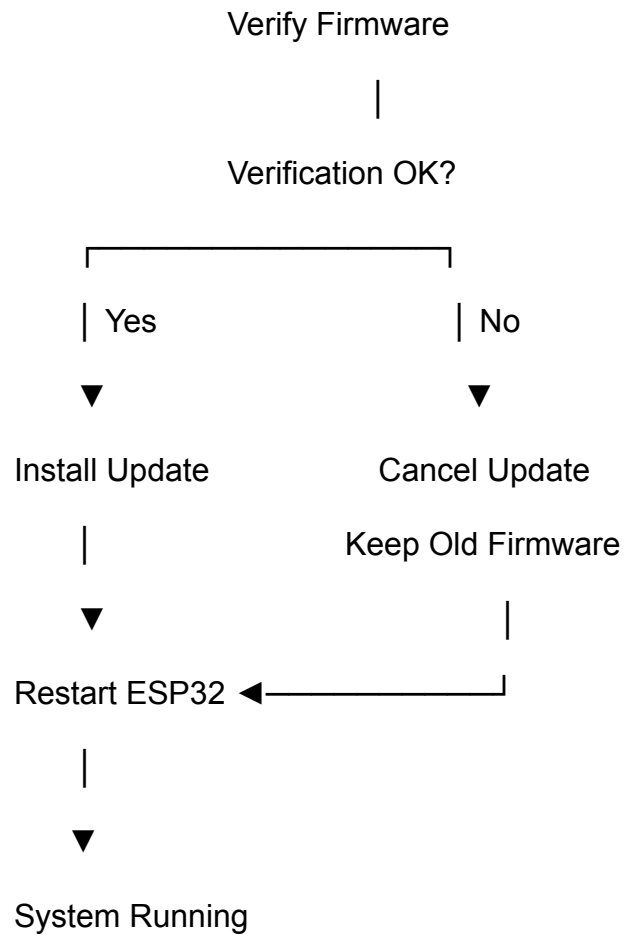
Firmware



Download

Firmware





Error Handling

- Verify firmware before installation.
- If download or verification fails, cancel the update.
- Continue running the previous firmware.
- Retry the update later.